Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Özyineleme (recursion) kavramı ve özyinelemeli algoritma tasarımı.
- Özyinelemeli bir yapının üç şartı.
- Hatalı özyineleme örneği: kendini koşulsuz çağıran `main`.
- Üs alma (power) örneği ve çağrıların izlenmesi.
- Özyinelemenin artıları ve eksileri: kod kolaylığına karşı zaman ve bellek maliyeti, fonksiyon çağrı yığını (function call stack).
- `mystery` fonksiyonu: basamakları tekrar tekrar toplama.
- Hanoi Kuleleri (Tower of Hanoi) problemi.
- Onluk sayıyı ikilik tabanda yazdıran özyinelemeli fonksiyon.
- Özyinelemeli algoritmaların analizi: yineleme denklemi (recurrence equation).
- Yineleme denklemlerini çözme yöntemleri; iterasyon (iteration) yöntemi.
- Özyinelemeli ikili arama ve $O(\log n)$ analizi.
- Özyinelemeli Fibonacci ve $O(2^n)$ analizi; özyineleme ağacı (recursion tree).

Hocanın Özellikle Vurguladığı Kısımlar

- Özyineleme kodlamayı çok kolaylaştırır ve kısaltır. Ama genellikle zaman karmaşıklığını ve bellek ihtiyacını artırır.
- Her özyinelemeli algoritmada kendini çağırmayan bir temel durum (base case) olmalıdır. Yoksa çağrılar sonsuza kadar sürer.
- Her özyinelemeli çağrı, problemi küçültmeli ve temel duruma yaklaştırmalıdır.
- Özyinelemeli çağrılar, her çağrının nereden yapıldığını ve parametrelerini saklamak için ek bellek ister. Çok derin özyineleme stack overflow'a yol açar.
- Özyinelemeli bir fonksiyon kendini birden fazla kez ve farklı parametrelerle çağırabilir. Örneğin Fibonacci $n-1$ ve $n-2$ ile çağırır.
- Masum görünen iki satırlık özyinelemeli Fibonacci kodu $O(2^n)$ karmaşıklığa sahiptir. Aynı iş basit bir döngüyle $O(n)$'de yapılır.
- Özyineleme her zaman iyi çözüm değildir. Fiyat-performans (trade-off) analizi yapılmalıdır.
- Hanoi Kuleleri'nin özyinelemeli algoritmasını kendiniz araştırıp çıkarın.

Kısa Tekrar Notları

- **Özyineleme:** Bir problemi aynı türden daha küçük alt problemlere indirip aynı çözümü küçük örnek üzerinde tekrar uygulamak.
- **Üç şart:**
  - Tüm geçerli girdileri karşılayan bir durum olmalı.
  - Özyineleme içermeyen bir temel durum olmalı.
  - Her özyinelemeli çağrı daha küçük ya da basit bir örneğe yapılmalı ve temel duruma doğru ilerlemelidir.
- **Yineleme denklemi:** Özyinelemeli algoritmanın süresi $T(n)$, kendi cinsinden yazılır. Örneğin $T(n) = c + T(n/2)$.
- **Çözüm yöntemleri:** iterasyon, yerine koyma (substitution), özyineleme ağacı (recursion tree), master yöntemi (master method).
- **Sık görülen yineleme denklemleri:**

Yineleme denklemi	Sonuç	Örnek
$T(n) = T(n-1) + c$	$O(n)$	Üs alma, `f(n)` yazdırma
$T(n) = T(n-1) + n$	$O(n^2)$	Her adımda n'lik döngü
$T(n) = T(n/2) + c$	$O(\log n)$	İkili arama
$T(n) = T(n/2) + n$	$O(n)$	Yarıya bölüp n'lik iş yapmak
$T(n) = 2T(n/2) + c$	$O(n)$	`g(n)` örneği
$T(n) = T(n-1) + T(n-2) + c$	$O(2^n)$	Özyinelemeli Fibonacci

Detaylı Açıklamalar

Özyinelemeli yapının şartları

- Özyinelemeli yapılar güçlü programlama araçlarıdır. Bazı önemli problemler için tek yol olabilirler.
- Fonksiyon kendini sürekli küçülen problemlerle çağırır. Temel duruma varınca çağrılar durur ve sonuçlar geriye doğru toplanır.

Hatalı örnek:

```c
int main() {
    main();
    return 0;
}
```

- Temel durum yok. Problem küçülmüyor. `main` kendini sonsuza kadar çağırır.
- Bu, özyinelemeli bir algoritmanın gerçeklemesi sayılmaz.

Üs alma

- Temel durum: $x^0 = 1$.
- Özyineleme: $x^n = x \cdot x^{n-1}$.
- $5^3 = 5 \cdot 5^2 = 5 \cdot 5 \cdot 5^1 = 5 \cdot 5 \cdot 5 \cdot 5^0 = 125$.

```c
int power(int x, int exp) {
    if (exp == 0)
        return 1;                    /* temel durum */
    else
        return x * power(x, exp - 1);
}

printf("%d", power(5, 3));
```

- Çağrıların izlenmesi:

Çağrı	Durum	Dönen değer
`power(5, 3)`	`5 * power(5, 2)`	125
`power(5, 2)`	`5 * power(5, 1)`	25
`power(5, 1)`	`5 * power(5, 0)`	5
`power(5, 0)`	temel durum	1

- Çağrılar yukarıdan aşağı yapılır, sonuçlar aşağıdan yukarı döner. En son çağrılan ilk işlenir.

Özyinelemenin artı ve eksileri

- **Artı:** Kod çok kısa ve akış basittir. `power` için tek bir fonksiyon yeter.
- **Eksi 1, zaman:** Genellikle döngüyle yazılan çözümden daha fazla zaman alır.
- **Eksi 2, bellek:** Her çağrıda "nereden geldim, hangi parametrelerle çağrıldım" bilgisi saklanır. Bu bilgiyi derleyici tutar; algoritmanın kendi verisi değildir ama ek bellek (overhead) gerektirir.

- Çok büyük girdilerde (örneğin çok büyük üsler) bellek sınırı nedeniyle program çöker.

`mystery` fonksiyonu

```
int mystery(int n) {
    int a, b;
    if (n < 10)
        return n;        /* temel durum */
    a = n / 10;
    b = n % 10;
    return mystery(a + b);
}
```

- `mystery(648)`: $a = 64$, $b = 8$, `mystery(72)` çağrılır.
- `mystery(72)`: $a = 7$, $b = 2$, `mystery(9)` çağrılır.
- `mystery(9)`: $9 < 10$, 9 döner. Bu değer hiç değişmeden en başa kadar taşınır. Sonuç 9.
- `power`'dan farkı: Burada dönen değer üzerinde ek işlem yapılmaz.

Hanoi Kuleleri

- Üç çubuk (A, B, C) var. A'da büyükten küçüğe dizili 5 disk var. Amaç kuleyi C'ye taşımak.
- **Kurallar:**
  - Her hamlede yalnız bir disk taşınır.
  - Büyük disk hiçbir zaman küçük diskin üstüne konamaz.
- Ara çubuk (B) yedek, yardımcı çubuk olarak kullanılır.
- **Özyinelemeli fikir:**
  - Üstteki 4 diski yedek çubuğa taşı.
  - En büyük diski hedef çubuğa taşı. Problem artık 4 diskli bir probleme indi.
  - 4 diski de aynı yöntemle hedefe taşı: 3 diski yedeğe al, 1 diski taşı, ve böyle devam et.
  - Tek diskli problem temel durumdur: Disk doğrudan hedefe taşınır.
- Algoritmada hangi diskin, hangi çubuktan, hangi çubuğa taşındığı ve yardımcı çubuk parametre olarak tutulmalıdır.

Onluk sayıyı ikilik tabanda yazdırma

- Fikir: Sayıyı sürekli 2'ye böl. Kalanlar ikilik basamaklardır.
- Özyineleme sayesinde basamaklar doğru sırada yazılır; ters çevirmeye gerek kalmaz. En anlamlı basamak en derindeki çağrıda yazılır.
- Temel durum: $n < 2$ ise n (0 ya da 1) yazdırılır.

```
void printInBinary(int n) {
    if (n < 2)
        printf("%d", n);          /* temel durum */
    else {
        printInBinary(n / 2);
        printInBinary(n % 2);
    }
}
```

- $n = 43$ için izleme:
  - Çağrılar: f(43), f(21), f(10), f(5), f(2), f(1).
  - f(1) "1" yazar.
  - Geri dönüşte sırayla f(2 mod 2) "0", f(5 mod 2) "1", f(10 mod 2) "0", f(21 mod 2) "1", f(43 mod 2) "1" yazar.
  - Ekranda: `101011` (43'ün ikilik karşılığı).
- Döngüyle yapılsaydı bir dizi doldurup tersten yazmak gerekirdi; kod daha uzun olurdu.

Özyinelemeli algoritmaların analizi

- Süre fonksiyonu $T(n)$, kendi cinsinden yazılır. Buna yineleme denklemi (recurrence equation) denir.
- Önce temel durumun, sonra özyinelemeli durumun maliyeti yazılır.

Örnek 1:

```c
void f(int n) {
    if (n > 0) {
        printf("%d", n);
        f(n - 1);
    }
}
```

- `f(4)` sırayla 4, 3, 2, 1 yazar. `f(0)`'da hiçbir şey yapmadan durur.
- $T(0) = a$, $T(n) = b + T(n-1)$.

Örnek 2:

```c
int g(int n) {
    if (n == 1)
        return 2;
    else
        return 3 * g(n / 2) + g(n / 2) + 5;
}
```

- $T(1) = c$, $T(n) = b + 2T(n/2)$.
- 3 ile çarpmak ya da 5 eklemek sabit maliyettir.
- İki çağrı aynı olmak zorunda değildir. Fibonacci'de biri $n-1$, diğeri $n-2$ ile çağrılır.
- Hoca bu iki denklemin sonucunu söylemedi; sonuçlar "Kısa Tekrar Notları"ndaki tabloda var.

Yineleme denklemlerini çözme yöntemleri:

- İterasyon yöntemi (iteration method).
- Yerine koyma yöntemi (substitution method).
- Özyineleme ağacı yöntemi (recursion tree method).
- Master yöntemi (master method): Belirli şartlar sağlanıyorsa sonucu doğrudan veren, ispatlanmış kurallar.
- Bu derste yalnız iterasyon yöntemi kullanılacak.

Özyinelemeli ikili arama

```c
int binarySearch(int A[], int x, int low, int high) {
    int mid;
    if (low > high)
        return 0;                                   /* bulunamadı (false), c1 */
    mid = (low + high) / 2;                         /* c2 */
    if (x == A[mid])
        return 1;                                   /* bulundu (true), c3 */
    else if (x < A[mid])
        return binarySearch(A, x, low, mid - 1);    /* T(n/2) */
    else
        return binarySearch(A, x, mid + 1, high);   /* T(n/2) */
}
```

- İki özyinelemeli daldan yalnız biri çalışır. En kötü durumda $T(n) = c + T(n/2)$.
- **İterasyon yöntemiyle çözüm:**
  - $T(n) = c + T(n/2)$
  - $= c + c + T(n/4)$
  - $= c + c + c + T(n/8)$
  - $= k \cdot c + T(n/2^k)$
- Temel duruma ulaşmak için $n = 2^k$ alınır. İki tarafın logaritması alınınca $k = \log n$ olur.

- $T(n) = c \log n + T(1) = c \log n + c'$. Sabitler atılır: $O(\log n)$.
- Ek bellek: Fonksiyon kendini $\log n$ kez çağırdığı için çağrı yığınında $O(\log n)$ yer tutulur.

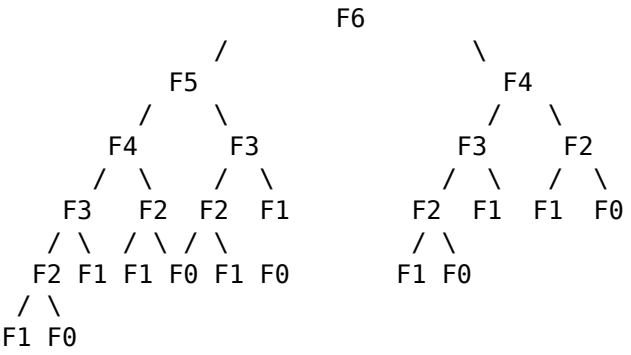
Özyinelemeli Fibonacci

- Dizi: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ... Her sayı kendinden önceki iki sayının toplamıdır.

```c
int fibonacci(int n) {
    if (n <= 1)
        return n;                                   /* temel durum */
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

- $T(0) = c_1$, $T(1) = c_2$, $T(n) = c + T(n-1) + T(n-2)$.
- **Üst sınır için yaklaşım:** n büyükken $T(n-2) \approx T(n-1)$ kabul edilir. Gerçekte $T(n-2) \leq T(n-1)$ olduğu için bu bir üst sınır verir.
- **İterasyon:**
  - $T(n) = 2T(n-1) + c$
  - $= 2(2T(n-2) + c) + c = 4T(n-2) + 3c$
  - $= 8T(n-3) + 7c$
  - $= 2^k T(n-k) + (2^k - 1)c$
- Temel durum için $n - k = 0$, yani $k = n$:
  - $T(n) = 2^n T(0) + (2^n - 1)c = 2^n(1 + c) - c$.
- Sonuç: $O(2^n)$.

Özyineleme ağacı (fib(6) örneği):



- Aynı alt problemler tekrar tekrar hesaplanır. F4 bloğu iki kez, F3 bloğu üç kez hesaplanır.
- Ağaç her seviyede yaklaşık iki katına çıktığı için çağrı sayısı 2^n mertebesine ulaşır.

Daha iyi çözüm:

- Bir dizide `F[0] = 0`, `F[1] = 1` ile başlanır. Sonra `F[i] = F[i-2] + F[i-1]` ile ilerlenir.
- Tek döngü: $O(n)$ zaman, $O(n)$ bellek.
- Yalnız son iki değer saklanırsa bellek sabit, yani $O(1)$ olur.

Sorulan sorular

- **Özyineleme neyi kazandırır?** Yalnız kodlama kolaylığı ve basit akış. Zaman ve bellekte genellikle kaybettirir.
- **Döngüyle aynı iş yapılamaz mı?** Hoca, `while` döngüsünde bir önceki adıma değer döndürülemediğini söyledi. Ama birçok problem döngüyle doğrusal bir çözüme çevrilebilir ve karmaşıklığı düşürdüğü için bu tercih edilir.
- **Özyinelemeli ikili arama döngüye göre bellek harcar mı?** Evet, çağrı yığınında $O(\log n)$ yer harcar.
- **Fibonacci belleği doldurur mu?** Evet, fonksiyon çağrı yığını büyür. Çok derin çağrılarda stack overflow oluşabilir.

Duyurular

- Gelecek hafta bağlı listeler (linked list) işlenecek.

Ders sonrası notlar

- `printInBinary` için bir öğrenci, ikinci özyinelemeli çağrı yerine doğrudan `printf("%d", n % 2);` yazılmasını önerdi. Hoca 11 sayısıyla izleme yaptı ve fazladan bir "1" yazılacağını söyledi. Bu sonuç yanlış: `f(1)` temel durumdur ve `else` kolu çalışmaz, yani orada `n % 2` yazılmaz.
  - Doğru izleme: f(11), f(5), f(2), f(1) çağrılır. f(1) "1" yazar. Geri dönüşte f(2) "0", f(5) "1", f(11) "1" yazar. Sonuç `1011`, 11'in doğru ikilik karşılığı.
  - Öğrencinin önerisi doğrudur ve bir fonksiyon çağrısını kurtarır:

```
void printInBinary(int n) {
    if (n < 2)
        printf("%d", n);
    else {
        printInBinary(n / 2);
        printf("%d", n % 2);
    }
}
```

- `printInBinary` iki çağrı yapsa da ikinci çağrı her zaman doğrudan temel duruma düşer. Bu yüzden karmaşıklık katlanmaz: $T(n) = T(n/2) + c$, yani $O(\log n)$.
- fib(6) ağacında hoca F2'nin 4 kez çağrıldığını söyledi. $n \leq 1$ temel durumuyla F5 1, F4 2, F3 3, F2 5, F1 8, F0 5 kez çağrılır. Toplam 25 çağrı yapılır.
- $T(n) = 2T(n-1) + c$ yaklaşımı bir üst sınırdır. Özyinelemeli Fibonacci'nin gerçek büyümesi $\Theta(\varphi^n)$'dir, $\varphi = (1+\sqrt{5})/2 \approx 1{,}618$. Bu da üsteldir.
- Hoca Fibonacci dizisini tahtaya yazarken bir "1"i atladı. Doğru dizi: 0, 1, 1, 2, 3, 5, 8, ...
- Bir öğrenci $1\,000\,000^{1\,000\,000}$ hesaplanırken çağrı yığınının ne olacağını sordu. Böyle bir hesapta 10^6 derinlikte özyineleme stack overflow'a yol açar. Sonuç zaten `int` ya da `long` türüne de sığmaz.
- `power` fonksiyonu $T(n) = T(n-1) + c$ ile $O(n)$'dir. $x^n = (x^{n/2})^2$ kuralını kullanan hızlı üs alma (fast exponentiation) $O(\log n)$'de çalışır.
- `mystery` fonksiyonu sayının basamak kökünü (digital root) bulur: Basamaklar tek basamak kalana kadar toplanır.
- Hanoi Kuleleri için hoca algoritmayı vermedi. Standart özyinelemeli çözüm:

```
void hanoi(int n, char kaynak, char hedef, char yardimci) {
    if (n == 1) {
        printf("Disk 1: %c -> %c\n", kaynak, hedef);
        return;
    }
    hanoi(n - 1, kaynak, yardimci, hedef);
    printf("Disk %d: %c -> %c\n", n, kaynak, hedef);
    hanoi(n - 1, yardimci, hedef, kaynak);
}
```

- Hanoi'de hamle sayısı $2^n - 1$'dir (5 disk için 31 hamle). Yineleme denklemi $T(n) = 2T(n-1) + 1$ olduğu için karmaşıklık $O(2^n)$'dir.
- Her özyinelemeli algoritma, açık bir yığın (stack) kullanılarak döngüyle de yazılabilir. Hoca "döngüde önceki adıma değer döndürülemez" derken doğrudan karşılığın olmadığını kastediyor.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.